

Mycelium 4.0

Rapport Final

BOTTOIS Elise - DUFOURCQ Thibault - LAHMAMSI Mohamed - LÉCRIVAIN Arno

Encadrants : PARLAVANTZAS Nikos - PAROL-GUARINO Volodia

Avec la participation de : MOUREAU Julien

2024/2025



Contents

1	Remerciements	3
2	Introduction	4
3	Rectificatifs des rapports	5
3.1	Scénario Inondation	5
3.2	Scénario Changement de fréquence	5
3.3	Proxy	6
4	Bilan de planification	6
5	Etat de finalisation du projet	7
5.1	Scénario Inondation	8
5.2	Scénario Changement de fréquence	8
5.3	Proxy	8
5.4	Ajout d'une Machine Virtuelle reproductible	9
6	Compte Rendu des phases de test	9
6.1	Scénario Inondation	10
6.2	Scénario Changement de fréquence	10
6.3	Proxy	11
7	Pistes pour la suite	11
8	Conclusion	12
9	Annexes	13

1 Remerciements

Nous tenons à exprimer notre profonde gratitude à M. Nikos PARLAVANTZAS et M. Volodia PAROLGUARINO, nos encadrants à l'INSA Rennes, pour leur accompagnement tout au long de ce projet. Leur disponibilité, leur expertise et leurs conseils avisés ont été déterminants pour surmonter les défis techniques et méthodologiques que nous avons rencontrés.

Nous remercions également chaleureusement M. Julien MOUREAU, notre référent à l'OSEREN, pour son aide précieuse dans la compréhension des enjeux scientifiques et techniques du projet.

Nous souhaitons enfin remercier les anciens membres du projet Mycélium 3.0 pour leur partage d'expérience. Nous adressons également une mention spéciale à Mohamed ALLAY, membre de notre équipe ayant contribué activement durant le premier semestre avant son départ en mobilité académique.

2 Introduction

Mycélium s’inscrit dans le cadre du projet national TERRAFORMA, une initiative visant à développer des dispositifs d’observation et d’expérimentation socio-écologiques des territoires pour encourager des transformations durables et équitables. Cette dynamique repose sur un constat clair : préserver l’habitabilité de la planète nécessite de mieux comprendre les environnements locaux. Dans ce cadre, des sites témoins sont mis en place, tels que celui de la Croix Verte (Figure 1), situé sur le campus de Beaulieu à Rennes. Ce site est particulièrement intéressant à suivre en raison des impacts écologiques liés à la construction récente de la ligne de métro sur cette zone.

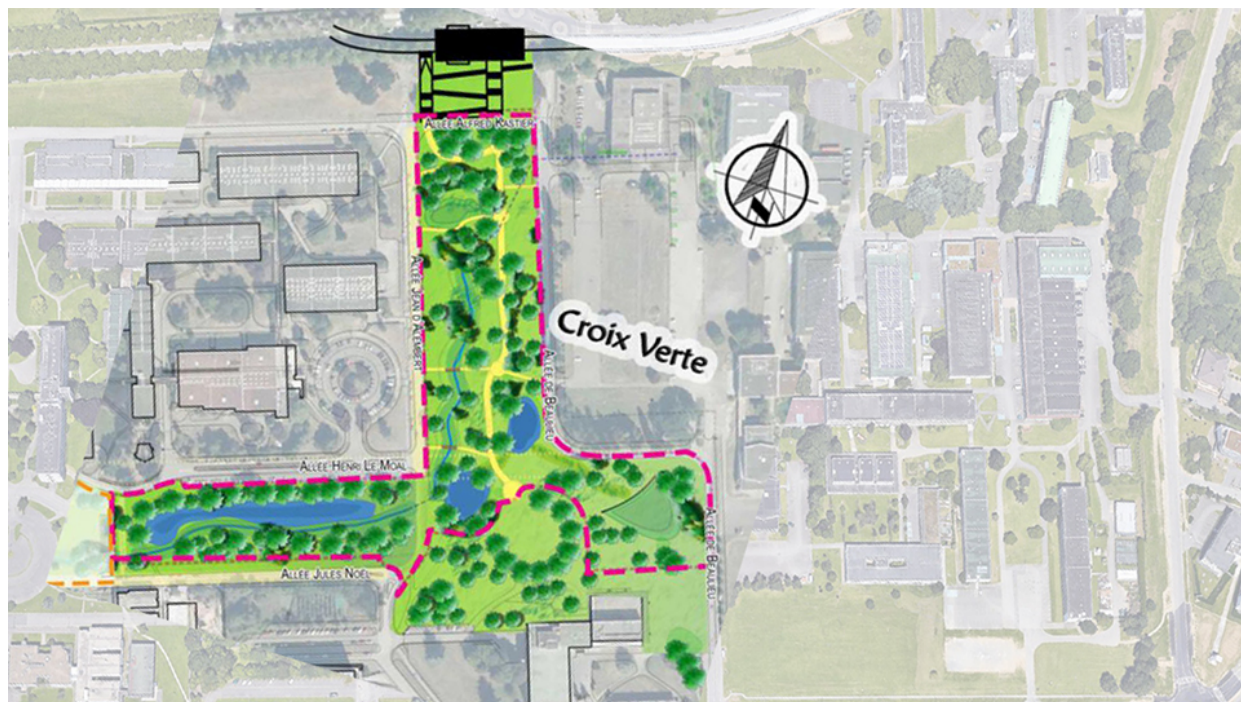


Figure 1: Carte du site de la Croix Verte sur le campus de Beaulieu.

Au-delà des enjeux environnementaux, sociaux et économiques, l’originalité du projet Mycélium réside dans les technologies déployées pour automatiser la collecte et le traitement des données environnementales. L’objectif est de s’affranchir au maximum des manipulations humaines grâce à l’utilisation d’un réseau de capteurs intelligents. Ces données, actuellement exploitées par les ingénieurs de l’OSEREN (partenaire principal du projet), ont pour vocation d’être à terme rendus publiques et Open Source, afin d’être accessibles aux acteurs locaux, scientifiques ou citoyens, qui pourront s’en emparer pour guider leurs décisions.

Depuis ses débuts, Mycélium a connu plusieurs itérations. Les versions 1.0 et 2.0 ont permis de poser les fondations techniques, avec la mise en place d’un réseau de capteurs basse consommation, le recours au protocole LoRaWAN [1], et l’introduction d’une architecture Fog Computing [2]. La version 3.0 a marqué une refonte importante visant à structurer le projet pour le rendre plus maintenable : un guide complet a été produit et une première machine virtuelle simulant un cluster a été créée.

La version 4.0, objet de ce rapport, poursuit cette logique d’amélioration continue. Elle introduit un environnement de développement simplifié et reproductible, intégrant notamment un proxy intelligent capable de gérer dynamiquement le routage des messages et les différentes versions de fonctions entre le cluster et le VPS [3]. De nouveaux scénarios concernant les inondations et le changement de fréquence

d'un capteur ont également été intégrés. L'un des objectifs majeurs de cette version est d'assurer une infrastructure plus stable, testable, et facilement transmissible aux futures équipes.

Ce rapport final présente les rectifications apportées sur les rapports précédents. Il dresse un bilan de la planification, fait état du travail accompli, et expose les réalisations ainsi que les perspectives pour les prochaines versions.

3 Rectificatifs des rapports

Au fil du développement des scénarios initialement définies dans le cahier des charges, il est rapidement apparu que certaines approches envisagées n'étaient pas adaptées aux contraintes réelles du projet, notamment en termes de faisabilité technique et de temps de développement. Cette section présente donc les ajustements apportés aux spécifications et à la conception du projet Mycelium, en réponse aux enjeux rencontrés tout au long de sa mise en œuvre.

3.1 Scénario Inondation

L'intégration des travaux réalisés par le groupe d'étude pratique de 3ème année sur le scénario inondation constituait un axe principal d'avancé cette année. L'objectif était de déployer l'architecture de ce projet dans l'environnement de travail virtuel de Mycélium dans un premier temps, puis sur le cluster, assurant ainsi une remontée d'alerte en cas d'inondation. Pour améliorer la pertinence de ces alertes, l'autre objectif relatif au scénario inondation était de réentraîner le modèle prédictif avec des données locales (plutôt qu'avec les données australiennes utilisées précédemment) afin d'en affiner les précisions : objectif que nous avons finalement abandonné. En effet, le temps de se synchroniser avec l'OSEREN pour obtenir les données, de les préparer puis d'adapter le modèle et de l'entraîner semblait trop long, ne nous permettant pas d'avancer au moment ni au rythme souhaité. Nous avons préféré nous focaliser sur l'intégration d'une adaptation propre et claire dans la VM pour les années suivantes. Ainsi, nous avons décidé de mettre de côté cet ajustement du modèle afin de nous concentrer sur l'objectif premier d'intégration du scénario à l'architecture générale.

3.2 Scénario Changement de fréquence

Dans le rapport de conception initial, le scénario du Changement de fréquence avait été présenté comme reposant sur une détection d'événements météorologiques anormaux à l'aide de tests statistiques tels que le test de Kolmogorov-Smirnov [4] et le test de Levene [5], ainsi que sur l'intégration d'un modèle de régression multiple pour la prédiction dynamique de la fréquence d'échantillonnage. Toutefois, lors de la mise en œuvre effective du projet, certaines modifications et simplifications ont été apportées, motivées par des considérations pratiques et techniques.

Initialement, il était prévu d'utiliser :

- Le test de Kolmogorov-Smirnov pour détecter des changements dans la distribution des données environnementales ;
- Le test de Levene pour surveiller les variations de la variance.

Cependant, dans l'implémentation finale, ce sont principalement des tests de Student [6] qui ont été utilisés.

Le test de Student permet de comparer les moyennes de deux échantillons de données pour détecter une différence significative. Ce choix a été motivé par plusieurs raisons :

- **Simplicité d'implémentation** : Le test de Student est plus simple à intégrer dans un système de traitement en temps réel, nécessitant moins de prétraitements complexes que les tests de Kolmogorov-Smirnov ou de Levene.

- **Pertinence par rapport aux objectifs** : L'objectif principal était de détecter rapidement et efficacement des changements significatifs des paramètres environnementaux (par exemple, augmentation de température). Le test de Student s'est révélé suffisant pour capturer ces variations de moyenne, qui correspondent bien aux types d'événements recherchés (pics de température, anomalies ponctuelles).
- **Performances et contraintes système** : Compte tenu des ressources limitées, l'utilisation de tests plus lourds comme Kolmogorov-Smirnov (KS) et Levene n'a pas été retenue. Ces tests nécessitent de conserver de grands volumes de données et de réaliser des calculs complexes, peu adaptés à un traitement en temps réel. En revanche, le test de Student, beaucoup plus léger, se contente de comparer les moyennes de deux échantillons à partir de formules simples. Son faible coût en mémoire et en calcul en fait une solution rapide, efficace et mieux adaptée aux contraintes de performance et d'efficacité énergétique du projet.

Ainsi, bien que le cadre théorique prévoyait une approche plus complète avec plusieurs types de tests, l'adaptation pragmatique au contexte opérationnel a conduit à privilégier l'efficacité et la simplicité.

La phase de conception envisageait également l'intégration d'un modèle de régression multiple pour ajuster dynamiquement la fréquence d'échantillonnage en fonction des conditions environnementales anticipées.

Cependant, ce modèle n'a finalement pas été implémenté pour plusieurs raisons :

- **Complexité et temps de développement** : La création, l'entraînement, la validation croisée et l'intégration d'un modèle de machine learning nécessitent un investissement temporel conséquent. Or, le projet devait respecter des délais de développement courts.
- **Priorité donnée à la stabilité du système** : La priorité a été donnée à l'implémentation d'une solution robuste et stable basée sur des méthodes statistiques plus faciles à mettre en place, plutôt que d'introduire une couche de complexité supplémentaire avec un modèle prédictif encore incertain.

En conséquence, l'ajustement de la fréquence d'échantillonnage s'est basé uniquement sur la détection d'anomalies par comparaison de moyennes via le test de Student, sans recours à des prédictions par apprentissage machine.

3.3 Proxy

L'architecture initiale du proxy a été profondément remaniée pour favoriser la modularité, la scalabilité et la flexibilité. Plutôt qu'un proxy monolithique, trois microservices [7] ont été déployés de chaque côté (Cluster et VPS) : un Router pour le routage, un Monitor pour la remontée d'informations, et un Invoker pour l'appel des fonctions.

Par ailleurs, la gestion avancée des données via InfluxDB [8] (buckets synchronisés, renvois différés, compression) a été écartée dans le cadre du produit minimal viable. L'objectif était de se concentrer sur la détection de déconnexion, le fonctionnement des fonctions des deux côtés et la vérification des mécanismes de routage.

4 Bilan de planification

Mycelium est un projet s'appuyant sur l'utilisation de technologies avancées, souvent complexes à maîtriser. Cette technicité a entraîné de nombreux défis techniques, rendant l'avancement du projet plus difficile et plus lent que prévu. L'une des principales difficultés a été le manque de documentation fiable sur les technologies employées, certaines d'entre elles étant encore très récentes et peu diffusées dans le grand public. En conséquence, lorsqu'un problème survenait, il était souvent difficile de trouver des ressources ou des guides adaptés pour le résoudre par nous-mêmes. Par exemple, la modification dynamique de la

fréquence d'un capteur directement via les fonctions OpenFaaS [9] s'est avérée particulièrement complexe. L'interfaçage entre MQTT [10], ChirpStack [11] et les capteurs pour transmettre correctement ce type de commande en temps réel est peu documenté, ce qui a nécessité de nombreux ajustements expérimentaux.

La planification initiale du développement du proxy incluait une base de données locale destinée à stocker des données intermédiaires. Cette solution a finalement été abandonnée, principalement par manque de temps. Il convient de noter que deux bases de données sont tout de même utilisées dans le système : une sur le cluster et une sur le VPS. Ces bases ne sont pas synchronisées, et aucun mécanisme n'a été mis en place pour assurer le partage des données entre environnements. Une fonction peut donc échouer si elle dépend de données générées sur l'autre infrastructure.

Si les estimations de charge horaire pour les tâches réalisées se sont révélées globalement correctes, plusieurs facteurs non anticipés ont pesé sur l'avancement du projet : la rédaction des rapports intermédiaires, bien plus chronophage que prévu, la complexité d'assimilation des nouvelles technologies utilisées, ainsi qu'une ambition initiale démesurée (avec une estimation totale de 1500 heures à cinq, puis quatre membres, ce qui s'est avéré irréaliste). De plus, le temps nécessaire pour remettre en service le cluster a été sous-estimé. Ces éléments combinés expliquent l'écart entre la planification initiale et la réalisation effective.

En particulier pour l'adaptation du scénario inondation, la planification aurait pu être revue à la hausse en volume horaire car, si nous avons pu finaliser l'adaptation du côté de l'environnement de travail virtuel, nous avons par contre manqué de temps pour les ajustements nécessaires à la compatibilité avec le cluster. Nous avons sous-estimé la seconde partie de la tâche.

Concernant la mise en place de l'environnement de développement simplifié, un nombre d'heures conséquent avait été initialement alloué à cette tâche. Toutefois, grâce au soutien de notre encadrant, qui a pris en charge certains aspects techniques hors de notre périmètre (notamment la maintenance de la machine virtuelle), cette étape a pu être accélérée. Cela nous a permis de nous concentrer davantage sur nos tâches spécifiques et de libérer du temps pour d'autres développements.

La planification initiale des ressources humaines a également été ajustée en cours de projet pour gagner en efficacité. À l'origine, Arno et Thibault étaient affectés au développement du proxy et à la gestion de la machine virtuelle, tandis qu'Elise et Amine travaillaient sur les scénarios d'inondation et de changement de fréquence. En cours de projet, cette répartition a été revue : Arno s'est concentré sur la partie bas niveau du proxy, Amine a pris en charge le scénario de changement de fréquence, et Elise et Thibault se sont focalisés sur le scénario d'inondation. Elise étant en 3ème année et Thibault en 4ème année, chacun suivait un rythme scolaire différent, avec des dates butoirs parfois alternées, leur permettant parfois de se relayer sur leur travail partagé. Cette réorganisation a permis de mieux capitaliser sur les compétences spécifiques de chacun et d'optimiser la répartition des tâches.

Enfin, conformément à notre méthodologie initiale, nous avons pleinement adopté une approche agile tout au long du développement. Cette organisation itérative, structurée autour de points de synchronisation hebdomadaires, d'échanges transparents sur les difficultés rencontrées, et d'ajustements progressifs en réponse aux contraintes techniques, s'est révélée particulièrement efficace. Si certains objectifs initiaux ont été révisés en cours de route, c'est précisément cette agilité qui nous a permis d'atteindre un seuil de livrable robuste, cohérent et exploitable, répondant aux attentes d'un produit minimum viable. Chaque membre de l'équipe a su jouer le jeu de cette méthodologie pour faire émerger une solution aboutie, intégrée et alignée avec les enjeux à long terme du projet. Ce résultat concret, fruit d'une dynamique collective et d'un pilotage rigoureux, valide pleinement la pertinence de notre choix organisationnel.

5 Etat de finalisation du projet

Cette section présente l'état d'achèvement des différents éléments développés au cours du projet. Elle détaille les travaux réalisés pour les scénarios fonctionnels, l'évolution de l'architecture logicielle avec l'introduction du proxy, ainsi que les outils mis en place pour faciliter le développement et la maintenance. Chaque sous-partie revient sur le degré de finalisation atteint et les choix techniques opérés.

5.1 Scénario Inondation

Le scénario inondation a pu être adapté à Mycélium pour venir l’enrichir d’une fonctionnalité pertinente assurant une remontée d’alerte en cas d’inondation. La chaîne de traitement des données du capteur permet de déclencher un processus de prédiction parallèle. Ainsi, lors de la réception des données, une vérification du niveau d’eau va être déclenchée, suivie d’une estimation à court terme de son évolution (régression linéaire à l’aide du modèle à ré-entraîner). Ces deux résultats sont alors notifiés sur un salon discord dédié. La communication entre les fonctions du scénario a bien été basculée complètement sur le protocole MQTT, permettant une harmonisation avec l’ensemble du projet et un déploiement sur l’environnement virtuel.

5.2 Scénario Changement de fréquence

En se basant sur les travaux réalisés par le groupe Mycelium 3.0, qui ont permis de développer un scénario capable d’extraire et de stocker des données météorologiques issues de capteurs, le scénario de changement de fréquence a pu être développé, finalisé et testé avec succès. Ce scénario repose sur un ensemble de fonctions spécifiques d’extraction et de traitement permettant d’assurer la détection d’événements environnementaux anormaux et l’adaptation dynamique de la fréquence d’échantillonnage des capteurs.

Le fonctionnement repose principalement sur :

- **Des fonctions extractions OpenFaaS en Go** : ces fonctions permettent de récupérer les messages JSON décodés des capteurs (Milesight EM300), transmis via *Chirpstack* sur le broker *MQTT*. Ces fonctions extraient les données importantes des messages, les stockent dans la base de données *InfluxDB*, et transmettent ensuite les données pertinentes (températures, humidité etc) aux fonctions de traitement qui doivent être déclenchées.
- **Des fonctions traitement OpenFaaS en Go et Python** : ces fonctions réalisent les tests statistiques nécessaires pour décider d’un changement de fréquence du capteur. Elles effectuent des tests (notamment des tests de Student) sur les données extraites afin de détecter des anomalies. En fonction des résultats obtenus, elles ajustent dynamiquement la fréquence d’échantillonnage du capteur et publient le résultat de ce traitement sur un *topic MQTT* dédié aux notifications.

Grâce à cette architecture modulaire, le scénario de changement de fréquence est désormais pleinement opérationnel, offrant une solution robuste et réactive pour l’adaptation des capteurs en fonction de l’évolution des conditions environnementales.

5.3 Proxy

Par rapport à la version 3.0, l’architecture interne du projet a évolué de manière significative avec l’introduction d’un proxy dédié. Ce dernier joue un rôle central dans la gestion du routage des messages, en les redirigeant vers le cluster ou le VPS, selon une politique de routage définie. Le proxy a été conçu pour être modulaire et extensible : il est notamment capable de prendre en charge différentes versions d’une même fonction. Cette capacité permet de réaliser du shadow testing, en envoyant simultanément des requêtes à plusieurs implémentations sans impacter le fonctionnement global du système, afin d’expérimenter, comparer et valider de nouvelles versions en conditions réelles.

Un schéma de l’architecture finale intégrant ce proxy est présenté à la figure 2.

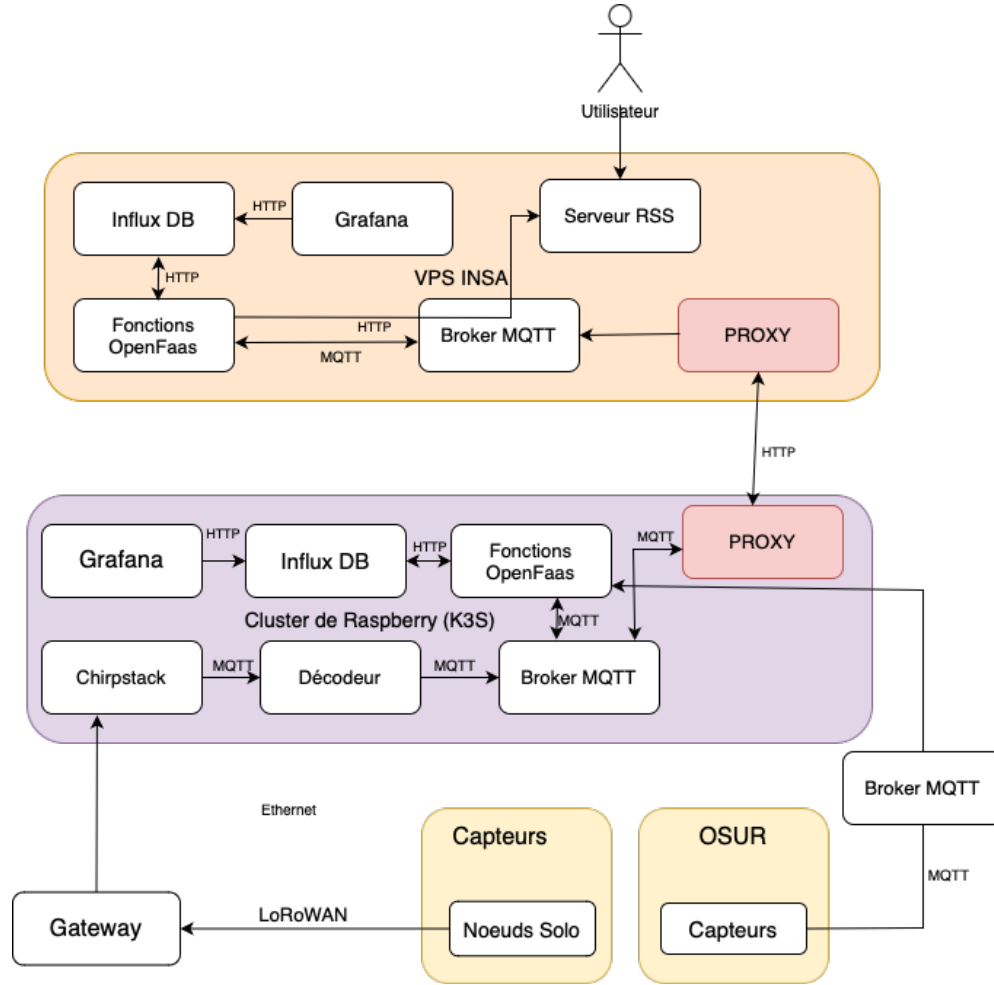


Figure 2: Architecture Matérielle de Mycelium 4.0

5.4 Ajout d'une Machine Virtuelle reproductible

Par ailleurs, une Machine Virtuelle complète a été construite à l'aide de Nix [12]. Elle simule les deux environnements principaux du projet (le Cluster Raspberry Pi et le VPS hébergé à l'INSA) et embarque une base de données, permettant ainsi un développement dans un contexte cohérent avec celui de la production. Cette VM, reproductible et facilement déployable, a grandement facilité les phases de test et de validation. Contrairement à la version utilisée dans Mycelium 3.0, elle a pu être utilisée sans encombre par plusieurs membres de l'équipe, garantissant un environnement de développement homogène et partagé. Elle est également conçue pour être réutilisable par les promotions suivantes, afin de capitaliser sur l'environnement mis en place cette année.

6 Compte Rendu des phases de test

Afin de valider la robustesse, la performance et l'efficacité énergétique des solutions développées, différentes phases de tests ont été menées tout au long du projet. Ces tests ont visé à vérifier le bon fonctionnement des scénarios fonctionnels, l'efficacité des algorithmes mis en place. Cette section détaille les méthodologies employées, les résultats obtenus, ainsi que les enseignements tirés des différentes campagnes de tests.

6.1 Scénario Inondation

Avec la création d’une fonction de simulation d’envoi de données par le capteur, nous avons pu vérifier la bonne transmission des informations par chaque fonction OpenFaas, ainsi que le bon déclenchement des alertes (inondation et prédiction), dès lors que la nature des données le nécessite. En écoutant sur chaque topic MQTT, on pouvait constater le passage des données, et la notification arrivait bien sur Discord, en adéquation avec l’observation initiale du capteur.

6.2 Scénario Changement de fréquence

Les tests réalisés dans le cadre du scénario de changement de fréquence visaient à valider la fonctionnalité et l’efficacité énergétique de l’approche d’adaptation dynamique de la fréquence d’échantillonnage des capteurs. Ces tests ont été menés en plusieurs étapes : tests unitaires, simulations comportementales basées sur des données historiques, puis analyse comparative de la consommation énergétique entre un mode classique (sans changement de fréquence) et notre mode adaptatif (avec changement de fréquence).

Chaque fonction du système a fait l’objet de tests unitaires permettant de s’assurer du bon fonctionnement des différents modules :

- **Test t de Student sur valeur unique** : évaluation de la pertinence de l’algorithme statistique pour comparer une nouvelle donnée à un ensemble historique, dans le but de détecter des écarts significatifs pouvant justifier une adaptation du comportement du capteur.
- **Publication MQTT** : vérification de la capacité du système à envoyer des messages de contrôle ou de notification vers des topics définis.
- **Changement de période de mesure** : test du mécanisme déclenchant une modification de l’intervalle d’échantillonnage (2 minutes en cas d’événement significatif, 15 minutes dans le cas contraire).

Une phase de simulation a ensuite été réalisée pour observer le comportement du système face à une variation brusque des conditions environnementales. Un jeu de données historiques simulées sur 24 heures (températures entre 23 et 25 °C, humidité autour de 50 %) a permis d’établir une base statistique de référence. Une nouvelle valeur fortement déviante (par exemple : 32 °C et 65 % d’humidité) a ensuite été introduite afin d’évaluer la réactivité du système.

L’algorithme applique un test t de Student unilatéral sur les mesures de température et d’humidité afin d’évaluer si une nouvelle donnée diffère significativement de la moyenne des observations précédentes. L’hypothèse nulle H_0 stipule qu’il n’y a *aucune différence significative* entre cette nouvelle valeur et les données historiques, ce qui reflète une stabilité des conditions environnementales.

- Si la *p-value* obtenue est inférieure à 0,05, H_0 est rejetée, suggérant un changement notable dans l’environnement. Le système réagit alors en augmentant la fréquence d’échantillonnage à 2 minutes afin de suivre de manière plus rapprochée cette évolution jugée significative.
- À l’inverse, si la *p-value* est supérieure ou égale à 0,05, H_0 est conservée. Cela indique qu’aucune variation significative n’a été détectée, et la fréquence d’échantillonnage reste inchangée (ou peut être réduite si elle avait été précédemment augmentée), ce qui permet d’optimiser l’utilisation des ressources système et énergétiques.

Dans ce jeu de données, l’algorithme a bien indiqué un changement de fréquence, bien évidemment, quand la nouvelle valeur est très similaire à l’historique, aucun changement de fréquence n’est annoncé.

L’adaptation dynamique de la fréquence d’échantillonnage constitue aussi un levier clé pour optimiser l’utilisation des ressources au sein du cluster. En ajustant la fréquence de collecte des données en fonction de la stabilité des conditions environnementales, on réduit directement le temps de sollicitation des processeurs, ce qui impacte favorablement l’ensemble de la consommation du système.

Il existe en effet une corrélation directe entre le temps d’activation du CPU et la consommation énergétique : moins le processeur est actif, plus la consommation diminue. Ce lien est particulièrement critique dans les environnements contraints, où chaque opération de traitement, de stockage ou de transmission pèse sur l’autonomie des équipements.

Réduire la fréquence d’échantillonnage durant les périodes calmes permet non seulement d’éviter le traitement inutile de données redondantes, mais également d’améliorer significativement l’efficacité énergétique et d’alléger la charge sur les ressources réseau et de stockage.

Cette approche offre ainsi un double avantage : assurer une surveillance fine et réactive lors des variations critiques, tout en maintenant une faible consommation énergétique en conditions stables. Elle se révèle particulièrement pertinente pour les réseaux à ressources limitées comme LoRaWAN, où l’autonomie des capteurs et la sobriété énergétique sont des priorités essentielles dans la conception des systèmes intelligents.

6.3 Proxy

Concernant le proxy, chacune de ses fonctionnalités a été testée localement à l’aide de fonctions basiques simulant des appels à différents topics MQTT. Ces tests ont permis de valider les politiques de routage, la récupération des informations de routage et d’utilisation du cluster, ainsi que l’invocation conditionnelle des fonctions en fonction des règles définies. Le comportement global a ainsi pu être vérifié en simulant divers scénarios d’utilisation.

En revanche, la mise en place de tests unitaires classiques pour chacun des microservices du proxy s’est révélée peu réaliste, car ceux-ci sont conçus pour fonctionner au sein d’un environnement orchestré (via Kubernetes [13] ou OpenFaaS). En dehors de cet environnement, les dépendances inter-services rendent les tests isolés complexes à automatiser et peu réaliste. Pour cette raison, les vérifications ont principalement été réalisées dans l’environnement de développement simulé par la machine virtuelle, permettant des tests plus proches de la réalité du déploiement.

7 Pistes pour la suite

Bien que le proxy soit fonctionnel dans sa version actuelle, plusieurs pistes d’améliorations peuvent être envisagées pour renforcer sa fiabilité, sa performance et sa scalabilité.

Une première évolution importante concerne la philosophie de récupération des politiques de routage. Actuellement, l’architecture repose sur une attente active, où le proxy interroge régulièrement le monitor via des requêtes HTTP GET pour récupérer les informations de routage. Cette méthode, bien que fonctionnelle, engendre des appels redondants et inutiles. L’évolution envisagée consiste à passer à un modèle plus réactif : le monitor enverrait des signaux au proxy via des requêtes POST uniquement lors de changements significatifs des politiques de routage. Cette approche permettrait de réduire la charge réseau, d’éliminer les requêtes superflues et de mieux désynchroniser les composants. Toutefois, puisque la récupération active par GET servait également à vérifier la disponibilité du monitor, cette nouvelle architecture nécessiterait la mise en place explicite d’un système de healthchecks pour surveiller son état, ainsi que des mécanismes de réessai en cas d’échec de transmission. Cela dit, avant d’envisager une telle refonte, il serait pertinent de mesurer précisément la bande passante consommée par le modèle actuel, afin d’évaluer si les gains attendus justifient réellement la complexité supplémentaire induite par cette évolution.

Par ailleurs, pour éviter toute perte de message en cas de surcharge ou d’indisponibilité temporaire, le processus d’invocation pourrait s’appuyer sur une solution de file d’attente. Plutôt qu’un système maison, l’usage de message queues comme RabbitMQ ou Kafka offrirait des garanties natives de persistance et de redelivery. À noter que MQTT propose déjà des niveaux de QoS (1 ou 2) permettant d’assurer une livraison fiable, ce qui pourrait suffire avec une configuration adaptée.

Enfin, bien que deux bases de données soient déjà présentes – l’une côté cluster et l’autre côté VPS – les fonctions d’un même scénario utilisant la base de donnée doivent actuellement être déployées dans le même environnement pour pouvoir accéder à la base de données correspondante. Une amélioration future pourrait

consister à mettre en place un mécanisme de synchronisation automatique des données entre les deux bases, permettant ainsi une exécution plus flexible des fonctions, indépendamment de leur lieu d'hébergement. Cela ouvrirait la voie à une véritable indépendance du déploiement, tout en garantissant l'accès aux données nécessaires.

Une dernière évolution envisageable serait d'intégrer au proxy une capacité d'adaptation dynamique à la charge du cluster. Ce mécanisme consisterait à transférer automatiquement certaines fonctions du cluster vers le VPS lorsque ce dernier devient trop sollicité, afin d'éviter les surcharges et de maintenir un bon niveau de performance. Une fois la charge stabilisée, ces fonctions pourraient être réintégrées au cluster, assurant ainsi une gestion intelligente et fluide des ressources. Cela nécessiterait une surveillance active de la charge et une orchestration fine, mais offrirait une importante résilience au système en conditions réelles.

Ces pistes ouvrent la voie à une version plus modulaire, performante et résiliente du proxy, mieux adaptée à un usage distribué en production.

Quant au scénario inondation, bien que son adaptation à l'environnement de travail virtuel ait été finalisée grâce à une uniformisation du protocole de communication (MQTT seulement), des ajustements seront nécessaires au niveau de la communication entre les nœuds du cluster pour le déploiement sur celui-ci. Par ailleurs, il conviendrait à l'avenir de ré-entraîner le modèle prédictif comme nous l'avions prévu initialement. En effet, même si notre arbitrage des priorités organisationnelles a mis cet objectif de côté pour l'instant, comme mentionné partie 3.1, il constituerait l'étape suivante logique afin de garantir des prédictions, et ainsi des alertes plus pertinentes et fiables. Le remplacement des données d'entraînement australiennes par des données locales permettrait cette amélioration.

8 Conclusion

En conclusion, ce projet s'est révélé particulièrement ambitieux, en s'inscrivant dans une démarche à la pointe de l'innovation technologique grâce à l'utilisation de solutions modernes comme OpenFaaS et Kubernetes. Notre engagement tout au long du projet nous a permis de développer des approches innovantes pour répondre aux nombreux défis techniques rencontrés et de fournir à la fin de ce cycle un projet qui répond aux attentes. Même si l'ensemble des objectifs initiaux n'a pas pu être entièrement atteint, l'expérience s'est avérée extrêmement formatrice. Elle nous a offert l'opportunité de nous familiariser avec des technologies émergentes, destinées à devenir incontournables dans les années à venir, notamment dans le cadre d'architectures de type Fog Computing, en pleine expansion.

References

- [1] Qu'est ce que le protocole LoraWan par WattSense. Disponible sur : <https://www.wattsense.com/fr-fr/blog/protocoles-de-communication/quest-ce-que-le-protocole-lorawan/>
- [2] Page Wikipedia Fog Computing. Disponible sur : https://en.wikipedia.org/wiki/Fog_computing
- [3] Qu'est ce qu'un VPS par Google Cloud. Disponible sur <https://cloud.google.com/learn/what-is-a-virtual-private-server?hl=fr>
- [4] INRIA, Test de Kolmogorov-Smirnov. Disponible sur : <https://mistis.inrialpes.fr/software/SMEL/cours/ts/node7.html>.
- [5] Wikipedia contributors, Levene's test. Disponible sur : https://en.wikipedia.org/wiki/Levene%27s_test.
- [6] Wikipedia contributors, Test de Student. Disponible sur : https://fr.wikipedia.org/wiki/Test_de_Student.
- [7] Que sont les microservices par Red Hat. Disponible sur : <https://www.redhat.com/fr/topics/microservices/what-are-microservices>
- [8] Documentation officielle d'InfluxDB. Disponible sur : <https://docs.influxdata.com/influxdb/v2/>
- [9] *Serverless Architecture with OpenFaaS and Java* par Xenonstack. Disponible sur : <https://www.xenonstack.com/blog/serverless-open-faas-java>
- [10] Documentation officielle de MQTT. Disponible sur : <https://mqtt.org/>.
- [11] Site Officiel de ChirpStack. Disponible sur : <https://www.chirpstack.io/>
- [12] Site Officiel de Nix. Disponible sur <https://nixos.org/>
- [13] Documentation officielle de Kubernetes. Disponible sur : <https://kubernetes.io/fr/docs/home/>

9 Annexes

- Lien du dépôt Gitlab contenant le Code du projet Mycelium 4.0 : <https://gitlab.insa-rennes.fr/mycelium-4.0/nixsed>
- Lien du Guide fourni par Mycelium 3.0, mis à jour : <https://docs.google.com/document/d/1zeqyLGFVsIeQFenrWVIXsBdJN7fD8fSSPS8Tly6i5KA/edit?tab=t.0#heading=h.8l6zs9xhhd2y>